

# 高级可视化

## Advanced Visualization

清华大学统计与数据科学系 · 俞声 · 2026.5.20

## 第一部分 图形语法 (Grammar of Graphics)

### 1.1 背景与起源

**图形语法 (Grammar of Graphics)** 是对统计图形构成要素的系统性形式化描述，由 Leland Wilkinson 于 1999 年在同名著作《The Grammar of Graphics》(Springer) 中提出。

Hadley Wickham 在 2010 年发表的论文 "A Layered Grammar of Graphics" 中，将该理论进一步发展分为分层语法，并实现为 R 语言的 ggplot2 包，成为数据可视化领域最具影响力的工具之一。Hadley 于 2019 年荣获 COPSS Presidents' Award (统计学领域最高荣誉之一)。

### 1.2 为什么需要形式化?

没有形式化体系时，每种图都是「特例」，需要分别学习对应的函数：

- 散点图 (Scatterplot)
- 柱状图 (Bar chart)
- 折线图 (Line chart)
- 直方图 (Histogram)
- 箱线图 (Boxplot)
- 热力图 (Heatmap)
- ..... 等等

**图形语法定义了一套通用语法**，将所有统计图形分解为可复用的组件：

| 组件                       | 示例                  |
|--------------------------|---------------------|
| Data (数据)                | 一个 DataFrame        |
| Aesthetic mapping (美学映射) | x = age, y = income |
| Geometry (几何图形)          | 点 (points)          |
| Scale (比例尺)              | 对数坐标轴               |

|                        |        |
|------------------------|--------|
| Coordinate system（坐标系） | 笛卡尔坐标系 |
|------------------------|--------|

形式化让隐藏的相似性得以显现，从而实现跨图形的知识复用与泛化。

## 第二部分 Plotnine 入门

### 2.1 什么是 Plotnine?

- **Plotnine** 是 R 语言 ggplot2 的 Python 移植版，基于图形语法设计。
- 语法与 ggplot2 几乎完全一致：统一、富有表达力、优雅。
- 绘图逻辑：先声明「想要什么」，plotnine 负责「如何画出来」。

### 2.2 安装

```
pip install plotnine
```

### 2.3 构建图形的三个基本步骤

无论多复杂的图形，都遵循以下三步：

- **第一步：**指定数据（data）和美学映射（aesthetics）
- **第二步：**添加一个或多个几何图形（geoms）
- **第三步：**添加标签（labels）和主题

### 2.4 第一个例子：教职人员薪资散点图

以教职薪资数据集（salaries.csv）为例，逐步构建一张散点图：

【步骤 1】读取数据，指定坐标轴映射：

```
import pandas as pd
from plotnine import *

data = pd.read_csv('salaries.csv').convert_dtypes()

# 此时只声明了数据和映射，尚未渲染
ggplot(data, aes(x='yrs.since.phd', y='salary'))
```

【步骤 2】加上几何图形（蓝色三角形散点）：

```
(ggplot(data, aes(x='yrs.since.phd', y='salary'))
 + geom_point(color='blue', shape='^', size=2.5))
```

### 【步骤 3】叠加拟合曲线：

```
(ggplot(data, aes(x='yrs.since.phd', y='salary'))  
+ geom_point(color='blue', shape='^', size=2.5)  
+ geom_smooth(color='red'))
```

### 【步骤 4】添加标题和轴标签：

```
(ggplot(data, aes(x='yrs.since.phd', y='salary'))  
+ geom_point(color='blue', shape='^', size=2.5)  
+ geom_smooth(color='red')  
+ labs(title='Faculty Salary', x='Years since PhD', y='Salary'))
```

#### 核心要点

图形是「逐层」构建的（语义层，而非渲染层）。层与层之间用重载的 '+' 运算符链接。注意：'+' 不满足交换律，顺序不同结果可能不同。图形只在最终打印时才渲染。

## 2.5 美学映射 (Aesthetics)

美学 (Aesthetics) = 图上可感知的视觉属性，例如：位置 (x/y)、颜色 (color)、填充色 (fill)、形状 (shape)、大小 (size)、透明度 (alpha) 等。

| 美学属性     | 作用        | 典型用途     |
|----------|-----------|----------|
| x / y    | 确定坐标位置    | 所有图形必需   |
| color    | 点/线/边框颜色  | 分组区分     |
| fill     | 填充颜色      | 柱/直方/密度图 |
| shape    | 点的形状      | 散点图分组    |
| size     | 点或线的大小    | 气泡图、强调   |
| alpha    | 透明度 (0~1) | 重叠区域可视化  |
| linetype | 线型        | 折线图区分    |

**关键规则：**由数据决定的属性写在 `aes()` 内部；固定常量写在 `aes()` 外部。

```
# 颜色由数据中的 sex 列决定 → 放在 aes() 内  
ggplot(data, aes(x='salary', color='sex')) + geom_density()  
  
# 颜色固定为蓝色 → 放在 aes() 外  
ggplot(data, aes(x='salary')) + geom_density(color='blue')
```

`aes()` 可以在 `ggplot()`（全局）或 `geom` 函数（局部覆盖）中设置。

## 2.6 Geoms（几何图形）

Geom 决定「用什么形状来呈现数据」。每个 geom 都有自己的可定制参数。

完整列表见：<https://plotnine.org/reference/#geoms>

| Geom 函数           | 用途                   |
|-------------------|----------------------|
| geom_point()      | 散点图                  |
| geom_line()       | 折线图                  |
| geom_smooth()     | 平滑拟合曲线（带置信区间）        |
| geom_bar()        | 柱状图（自动计数）            |
| geom_col()        | 柱状图（使用已有 y 值，基于 x 轴） |
| geom_histogram()  | 直方图                  |
| geom_density()    | 核密度估计曲线              |
| geom_boxplot()    | 箱线图                  |
| geom_violin()     | 小提琴图                 |
| geom_jitter()     | 抖动散点图（处理重叠）          |
| geom_density_2d() | 二维密度等高线              |
| geom_bin_2d()     | 二维 bin 热力图           |
| geom_text()       | 文字标注                 |
| geom_abline()     | 由斜率和截距指定的直线          |

## 2.7 常用图形示例

### 直方图（Histogram）

```
# 教职薪资直方图
ggplot(data, aes(x='salary')) + geom_histogram(fill='blue', alpha=0.3)
```

### 散点图 + 平滑曲线

```
# 薪资 vs 工龄，附拟合曲线
(ggplot(data, aes(x='yrs.service', y='salary'))
 + geom_point()
 + geom_smooth(color='red'))
```

### 箱线图 + 抖动点 + 小提琴图

```
# 按职级分组的薪资分布
(ggplot(data, aes(x='rank', y='salary'))
 + geom_violin(fill='lightblue'))
```

```
+ geom_boxplot(width=0.2)
+ geom_jitter(width=0.1, alpha=0.4))
```

## 柱状图（计数 vs 均值）

```
# 各职级人数 (geom_bar 自动计数)
ggplot(data, aes(x='rank')) + geom_bar()

# 各职级平均薪资 (需先计算, 再用 geom_col)
avg = data.groupby('rank', as_index=False)['salary'].mean()
ggplot(avg, aes(x='rank', y='salary')) + geom_col()
```

## 2.8 颜色与形状

颜色和形状既可以静态设置，也可以通过 `aes()` 由数据驱动（同时自动生成图例）：

```
# 按职级颜色编码 + 形状编码
(ggplot(data, aes(x='yrs.service', y='salary', color='rank', shape='rank'))
 + geom_point(size=2))

# 按性别分组的密度图
(ggplot(data, aes(x='salary', fill='sex', color='sex'))
 + geom_density(alpha=0.3))
```

## 2.9 分面（Faceting）

当分组数据叠在一张图里显得拥挤时，用分面将各组绘制在独立的子图中，关系更清晰。

| 函数                        | 适用场景         | 参数                                                |
|---------------------------|--------------|---------------------------------------------------|
| <code>facet_grid()</code> | 1~2 个变量，整齐网格 | <code>rows='变量名'</code> , <code>cols='变量名'</code> |
| <code>facet_wrap()</code> | 1 个变量、水平较多时  | <code>nrow=行数</code> , <code>ncol=列数</code>       |

```
# 按性别分面，散点图颜色按职级编码
(ggplot(data, aes(x='yrs.service', y='salary', color='rank'))
 + geom_point()
 + facet_grid(cols='sex'))

# 按职级 × 性别做网格分面
(ggplot(data, aes(x='salary'))
 + geom_density()
 + facet_grid(rows='rank', cols='sex'))

# facet_wrap 示例
(ggplot(data, aes(x='salary', fill='rank'))
 + geom_density(alpha=0.3)
 + facet_wrap('rank', nrow=1))
```

## 2.10 比例尺 (Scales)

Scales 控制数据到美学属性的映射方式，最常见的用途是自定义颜色映射。

| 函数类型                                                    | 用途                     |
|---------------------------------------------------------|------------------------|
| <code>scale_color_brewer(type=, palette=)</code>        | 离散颜色 (ColorBrewer 色板)  |
| <code>scale_fill_brewer(type=, palette=)</code>         | 离散填充色 (ColorBrewer 色板) |
| <code>scale_color_distiller(type=, palette=)</code>     | 连续颜色渐变 (ColorBrewer)   |
| <code>scale_color_manual(values=[...])</code>           | 手动指定颜色                 |
| <code>scale_x_continuous(breaks, labels, limits)</code> | 连续 x 轴刻度/标签/范围         |
| <code>scale_y_discrete()</code>                         | 离散 y 轴                 |

```
# 更改颜色映射 (按职级, 使用 Set1 色板)
(ggplot(data, aes(x='yrs.service', y='salary', color='rank'))
 + geom_point()
 + scale_color_brewer(type='qual', palette='Set1'))

# 按薪资高低连续着色
(ggplot(data, aes(x='yrs.service', y='salary', color='salary'))
 + geom_point()
 + scale_color_distiller(type='seq', palette='Blues', direction=1))

# 修改 x 轴刻度
(ggplot(data, aes(x='salary'))
 + geom_histogram()
 + scale_x_continuous(breaks=[50000,100000,150000,200000],
                      labels=['5 万','10 万','15 万','20 万']))
```

## 2.11 标签与主题

```
# labs() 添加标题、轴标签、图例标题
labs(title='教职薪资分析', x='博士毕业年数', y='薪资 (美元)', color='职级')

# 内置主题
theme_gray()      # 默认主题
theme_bw()        # 黑白主题
theme_minimal()   # 极简主题
theme_classic()   # 经典主题

# 自定义主题元素
theme(axis.text.x=element_text(angle=45, hjust=1),
      legend.position='bottom')
```

## 2.12 保存图形

```
# 保存为 PNG (栅格, 适合演示)
p.save('my_plot.png', width=8, height=6, dpi=300)

# 保存为 PDF (矢量, 适合论文/印刷)
p.save('my_plot.pdf', width=8, height=6)
```

*width/height 单位为英寸, dpi 影响 PNG 文件的像素密度。PDF 为矢量格式, 无需设置 dpi。*

## 第三部分 交互式数据应用 (Streamlit)

### 3.1 工具选择

构建交互式 Python 数据应用的主流框架:

| 框架             | 定位              |
|----------------|-----------------|
| Django / Flask | 高并发生产应用, 复杂需求   |
| Streamlit      | 最适合数据科学仪表盘、内部工具 |
| Dash           | 同样适合数据应用, 偏前端定制 |
| Gradio         | 快速演示机器学习模型      |

### 3.2 什么是 Streamlit?

- Python 模块, 让数据科学家用极少代码构建交互式 Web App。
- 无需 HTML / JavaScript, 只写 Python。
- **运行方式:** 在终端执行 `streamlit run app.py`, Streamlit 启动本地服务 (默认端口 8501) 并在浏览器中打开。
- **停止服务:** 按 `Ctrl + C` (关闭浏览器不会停止服务, 因为 Streamlit 在后端独立运行)。

### 3.3 第一个 Streamlit App

页面元素按代码从上到下的顺序渲染。每次修改代码后, 网页自动刷新。

```
import streamlit as st
```

```
st.title('My First Streamlit App')

name = st.text_input(label="What's your name?", value='your name')
age = st.slider(label='Age', min_value=0, max_value=100, value=20)

st.write(f'{name} ({age})')
```

### 3.4 响应式是如何实现的？

机制极其简单：

**核心机制**  
每当用户与 App 中的小组件（widget）交互，Streamlit 就重新运行整个脚本。当前 widget 的返回值就是用户的最新输入。

### 3.5 Widgets（交互控件）

Widgets 语法统一，都由 label（说明文字）+ 控件特定参数 + 初始值组成：

```
name = st.text_input(label="What's your name?", value='your name')
age = st.slider(label='Age', min_value=0, max_value=100, value=20)
#           ↑ 说明文字           ↑ 控件特定参数           ↑ 初始值
```

常用 Widgets 一览：

| Widget             | 用途       |
|--------------------|----------|
| st.text_input()    | 单行文本输入   |
| st.text_area()     | 多行文本输入   |
| st.number_input()  | 数字输入     |
| st.slider()        | 滑块（数值范围） |
| st.selectbox()     | 下拉单选     |
| st.multiselect()   | 下拉多选     |
| st.radio()         | 单选按钮     |
| st.checkbox()      | 复选框      |
| st.button()        | 按钮       |
| st.date_input()    | 日期选择     |
| st.file_uploader() | 文件上传     |

Widget 返回值是普通 Python 变量，可直接用于后续计算和绘图。



## 3.6 输出元素

Streamlit 提供多种输出元素。可使用万能的 `st.write()`，也可指定具体函数：

```
st.write('普通文本或任意对象')          # 万能输出
st.title('页面主标题')
st.header('一级标题')
st.subheader('二级标题')
st.markdown('**加粗**、*斜体*、LaTeX:  $x^2$ ')
st.dataframe(df)                          # 交互式表格
st.table(df)                              # 静态表格
st.metric('指标名', value=42, delta=+3)   # 指标卡片
st.json({'key': 'value'})                 # JSON 展示
st.code('print(42)', language='python')   # 代码展示
```

支持 LaTeX 数学公式：

```
st.markdown(r'''
### 二次方程求根公式

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

''')
```

## 3.7 绘图集成

Streamlit 内置交互式图表，也支持 matplotlib（及所有基于它的库，如 seaborn、plotnine）：

```
import streamlit as st
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from plotnine import *

data = pd.DataFrame(np.random.randn(20, 3), columns=['a', 'b', 'c'])

# Streamlit 内置图表（交互式）
st.bar_chart(data)
st.line_chart(data)

# matplotlib 图
fig, ax = plt.subplots()
ax.plot(data)
st.pyplot(fig)

# plotnine 图（先 draw() 转为 matplotlib Figure）
df = pd.DataFrame({'x': np.tile(np.arange(20), 3),
                   'y': data.values.reshape(60),
                   'g': np.repeat(['a', 'b', 'c'], 20)})
p = ggplot(df, aes(x='x', y='y', color='g')) + geom_line()
```

```
fig = p.draw()
st.pyplot(fig)
```

### 3.8 缓存（Caching）

每次 widget 交互都会重新运行整个脚本。对于耗时操作（如读取大文件、调用 API），应使用缓存避免重复执行：

| 装饰器                | 适用场景                     |
|--------------------|--------------------------|
| @st.cache_data     | 缓存返回数据的函数（DataFrame、列表等） |
| @st.cache_resource | 缓存全局资源（ML 模型、数据库连接等）     |

```
@st.cache_data
def load_data(url):
    return pd.read_csv(url)

# 文件只在第一次调用时读取，之后直接用缓存结果
data = load_data('xiehe_concepts_20260505.txt')

num = st.slider('How many samples?', 5, 100, 30)
st.dataframe(data.sample(num))
```

### 3.9 Session State（会话状态）

由于每次交互都重新运行脚本，普通变量的值会丢失。Session State 提供字典接口，在多次脚本运行之间保留数据：

```
# 经典计数器示例
if 'counter' not in st.session_state:
    st.session_state['counter'] = 0

st.session_state['counter'] += 1

st.header(f"This page has run {st.session_state['counter']} times.")
st.button('Run it again')
```

### 3.10 用 Session State 构建 AI 聊天机器人

大语言模型（LLM）的多轮对话机制与单轮问答本质相同：将历史所有对话（附带 role 标签）拼接进 prompt，模型生成下一条回复。

| 模式 | 说明 |
|----|----|
|----|----|

|                    |                                                               |
|--------------------|---------------------------------------------------------------|
| 单轮 QA (completion) | LLM 根据 prompt 直接生成文本，适合一次性问答                                  |
| 多轮对话 (chat)        | 将所有历史消息 (user/assistant 交替) 作为 context 传入，模型扮演 assistant 继续生成 |

本质上，*chat* 和 *completion* 已统一为同一个 API，差别只在消息格式。

使用 Session State 存储对话历史，实现多轮聊天：

```
import streamlit as st
from openai import OpenAI # 或其他 LLM SDK

client = OpenAI()

# 初始化对话历史
if 'messages' not in st.session_state:
    st.session_state.messages = []

st.title('我的聊天机器人')

# 显示历史消息
for msg in st.session_state.messages:
    with st.chat_message(msg['role']):
        st.markdown(msg['content'])

# 接收用户输入
if prompt := st.chat_input('请输入问题...'):
    # 保存用户消息
    st.session_state.messages.append({'role': 'user', 'content': prompt})
    with st.chat_message('user'):
        st.markdown(prompt)

    # 调用 LLM 生成回复
    response = client.chat.completions.create(
        model='gpt-4o',
        messages=st.session_state.messages # 传入完整历史
    )
    reply = response.choices[0].message.content

    # 保存并显示 AI 回复
    st.session_state.messages.append({'role': 'assistant', 'content': reply})
    with st.chat_message('assistant'):
        st.markdown(reply)
```

## 附录 参考资料

- ggplot2 官方文档: <https://ggplot2.tidyverse.org/reference/>
- Plotnine 官方文档: <https://plotnine.org/reference/>
- Streamlit 官方文档: <https://docs.streamlit.io/>
- Wilkinson, L. (1999). The Grammar of Graphics. Springer.
- Wickham, H. (2010). A Layered Grammar of Graphics. Journal of Computational and Graphical Statistics.

